

Reflection Template

Use Case	UC-6
Use Case Name	Create Task
Requirement IDs	FR5, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-13 – CS-16
Implementation Order	Kotlin first -> Java second

Feature Relevance

Feature/s:

Null safety, data classes/records, extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, the Task entity has several optional fields (description, dueDate, assignedUser) which made null safety very visible, especially in the mapper where assignedUser could be null. Kotlin's safe call operator (?.) handled this naturally in one expression, while Java needed an explicit null check. Data classes and records were used the same way as before. The extension function and RepositoryUtils were both reused without changes.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	3	5
Compactness of logic	3	5
Perceived implementation effort	3	5

Justifications

Readability:

In the mapper the difference is most visible. Kotlin's task.assignedUser?.id and task.assignedUser?.name are immediately clear. Java's ternary operator assignedUser != null ? assignedUser.getId() : null is more verbose and has to be repeated for each nullable field. The entity in Java also requires a lot more code with getters and setters for every field.

Compactness of logic:

Java's Task entity is the longest file so far; getters, setters, and constructors for every field add up quickly. Kotlin's entity is much shorter. The mapper difference is also meaningful here, two safe call expressions vs. two ternary operators.

Implementation effort:

Java took noticeably longer due to the entity boilerplate. The mapper also required more thought since null handling had to be written explicitly for each nullable field.

Observed structural differences

Kotlin's safe call operator (?.) handles nullable fields in a single expression `task.assignedUser?.id`. Java requires an explicit null check for each field `assignedUser != null ? assignedUser.getId() : null`.

Nullable fields in `CreateTaskRequest` and `TaskResponse` are explicitly marked with `?` in Kotlin. In Java the same fields are just regular types with no compile-time indication of nullability.

The Task entity in Java is significantly longer than in Kotlin due to getters, setters, and constructors. This is the most boilerplate-heavy entity so far.

Both services use their respective utility `RepositoryUtils.findByIdOrThrow` in Java and `findByIdOrThrow` as an extension function in Kotlin. The call syntax difference remains consistent like in UC5.

Final comparative summary

Here is where null safety has the most visible impact so far. Multiple optional fields in the entity, request and response make the difference between Kotlin's type-level nullability and Java's implicit nullability very concrete. The safe call operator keeps the mapper compact in Kotlin, while Java requires repeated explicit null checks. The difference in amount of code in entities is also at its peak here.